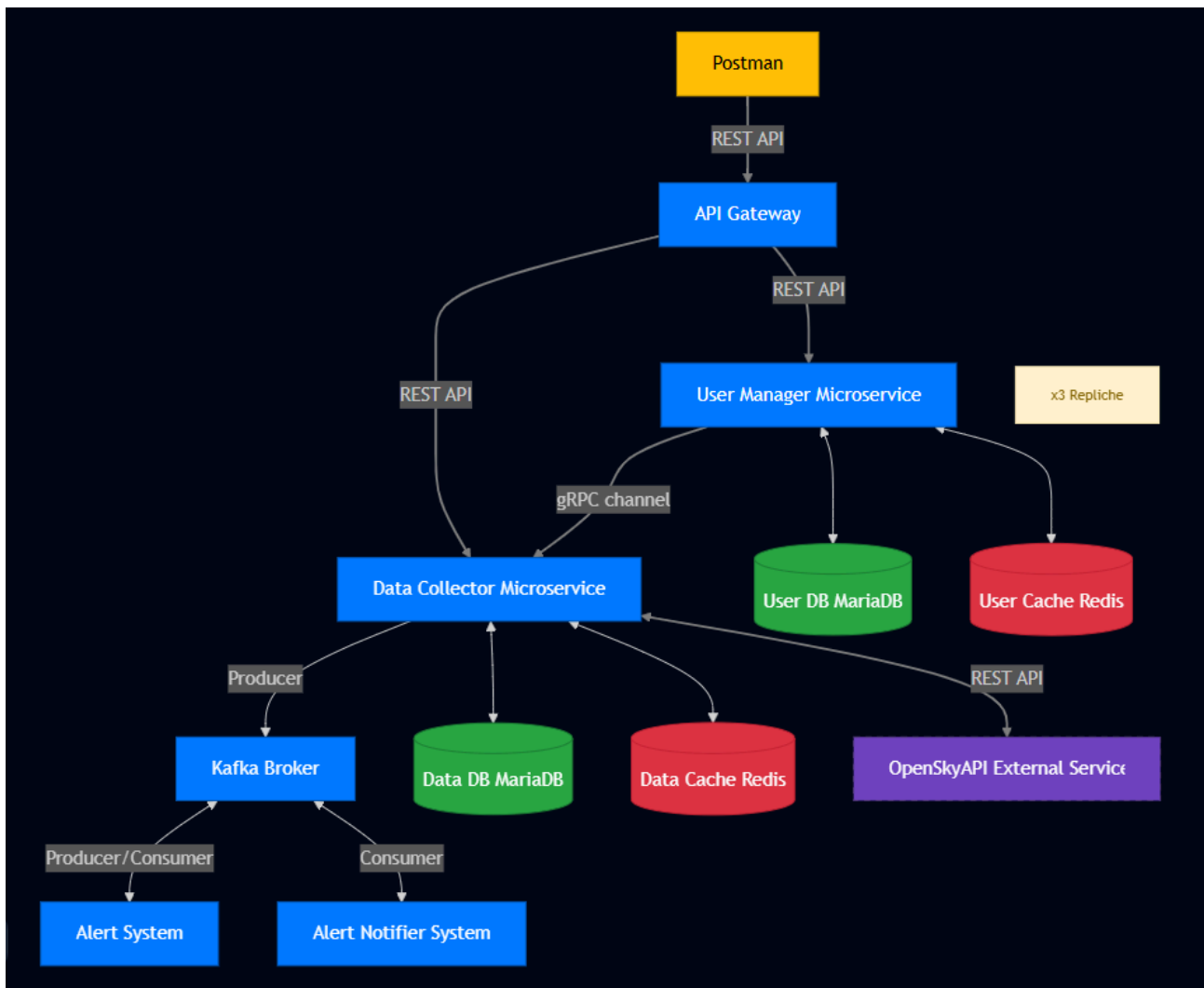


DISTRIBUTED SYSTEM & BIG DATA

HOMEWORK2

DANIELE DI PRIMO – MATTEO SECONDO

1. Panoramica dell'Architettura



Il progetto si è evoluto verso un'architettura a microservizi avanzata, che integra pattern di **API Gateway** e **Event-Driven Architecture** per garantire scalabilità, resilienza e un disaccoppiamento ancora più netto tra i componenti.

Il sistema è strutturato in tre layer logici principali:

A. Layer di Ingresso e Orchestrazione L'accesso dall'esterno non avviene più direttamente verso i singoli microservizi, ma è mediato da un **API Gateway**. Questo componente agisce come unico punto di ingresso (*Single Entry Point*) per il client (Postman), gestendo il routing delle richieste verso i servizi di backend appropriati e nascondendo la complessità della topologia interna.

B. Layer dei Core Microservices Questo livello gestisce la logica di business sincrona ed è composto da:

- **User Manager:** Responsabile della gestione delle identità. Per garantire alta disponibilità e gestire carichi elevati, questo servizio è stato scalato orizzontalmente con una configurazione a **3 repliche** attive.
- **Data Collector:** Responsabile dell'interazione con OpenSky Network per il reperimento dei dati di volo. Oltre a rispondere alle richieste dirette, ora agisce come *Producer*, immettendo i dati elaborati nel sistema di messaging.
- **Comunicazione:** L'API Gateway comunica con i due servizi tramite REST, mentre la comunicazione diretta tra User Manager e Data Collector continua a beneficiare delle performance di **gRPC**.

C. Layer Event-Driven (Asincrono) Per l'elaborazione dei dati e la gestione degli allarmi in tempo reale, è stato introdotto un sistema di messaggi basato su **Apache Kafka**. Questo abilita un flusso di dati asincrono:

- **Kafka Broker:** Riceve i flussi di dati dal Data Collector e li distribuisce ai consumatori interessati.
- **Alert System:** Un nuovo microservizio che si sottoscrive ai topic Kafka per analizzare i dati di volo in tempo reale e determinare, in base a regole di business, se attivare un allarme.
- **Alert Notifier System:** Un componente dedicato esclusivamente alla gestione delle notifiche tramite email verso gli utenti, attivato dagli eventi presenti su Kafka.
- **Kafka UI:** Espone un'interfaccia che permette la visualizzazione dei topic utilizzati, dei messaggi in arrivo, e dei consumer che sono sottoscritti ad essi.

L'adozione di Kafka e la separazione dei sistemi di Alert garantisce che l'elaborazione pesante e l'invio delle notifiche non impattino sulle prestazioni dei servizi principali rivolti all'utente.

2. Descrizione dei Moduli

A. API Gateway

Componente che funge da *Single Entry Point* per l'intera architettura, mascherando la complessità della rete interna. È strutturato su due moduli di configurazione principali:

- **Nginx Web Server:** Gestisce le regole di *Routing* basate sui percorsi (Path-based routing). Intercetta le richieste HTTPS (se arriva una richiesta HTTP viene fatto il redirect verso la corrispondente in HTTPS) in ingresso e, analizzando il prefisso dell'URL, determina verso quale microservizio inoltrare la chiamata (instradando tutto ciò che inizia con **/users** al servizio utenti e tutto ciò che inizia con **/flights** al data collector).
- **Load Balancer & gRPC Mediator:** Gestisce la distribuzione del carico per garantire che le repliche del servizio utenti siano utilizzate in modo efficiente.
 - **Bilanciamento HTTP:** Distribuisce le richieste REST esterne in ingresso tra le **3 repliche** attive dello *User Manager* utilizzando un algoritmo **Round Robin**.
 - **Mediazione gRPC (Traffico Interno):** Agisce da intermediario per la comunicazione gRPC interna avviata dal *Data Collector*. Invece di collegarsi staticamente a una singola replica, il

Data Collector stabilisce il canale gRPC direttamente con l'API Gateway. Quest'ultimo **smista dinamicamente la richiesta a una delle tre repliche** dello *User Manager*, la quale elabora il task e fornisce la risposta al Data Collector. Questo meccanismo previene colli di bottiglia su singole istanze e garantisce l'alta disponibilità anche per i processi interni.

B. User Manager Service

Microservizio deputato alla gestione del ciclo di vita delle utenze, deployato in più repliche per alta disponibilità.

- Endpoint Esposti (prefisso gateway: `/users`):
 - `POST /register`: Registrazione di una nuova anagrafica utente (input: email, nome, cognome).
 - `DELETE /delete`: Rimozione di un utente dal sistema tramite email.
- Componenti Dati:
 - User DB: Database relazionale (MariaDB) contenente la tabella anagrafica (PK: email).
 - User Cache: Istanza Redis utilizzata per ottimizzare le letture e garantire l'idempotenza delle scritture.

C. Data Collector Service

Microservizio core per la gestione dei dati aerei. Oltre a rispondere alle richieste REST, esegue task asincroni per il fetch dati da OpenSky e agisce come Producer verso il cluster Kafka, alimentando il sistema di allertamento.

- Endpoint Esposti (prefisso gateway: `/flights`):
 - `POST /airport-of-interest/add`: Sottoscrizione agli aggiornamenti per specifici aeroporti. Permette di definire opzionalmente le soglie `high_value` e `low_value`. A seguito dell'acquisizione dei dati, l'endpoint pubblica un evento sul topic Kafka; se il numero di voli rilevati supera la soglia massima o scende sotto la minima impostata, il sistema attiverà l'invio di una notifica email all'utente.
 - `GET /get-flights/latest`: Restituzione dell'ultimo volo rilevato (arrivo o partenza) per un dato aeroporto.
 - `GET /airport-of-interest/average`: Calcolo statistico (media voli e totali) su una finestra temporale definita.
- Componenti Dati:
 - Data DB: Database relazionale con tabelle `Flights` e `AirportsOfInterest`.
 - Data Cache: Istanza Redis segmentata (Email Check e Requests Cache) per garantire efficienza e ridurre il carico sulle API esterne.

3. Scelte Progettuali e Tecnologie

Il sistema è containerizzato con **Docker** e orchestrato via **Docker Compose**, che gestisce volumi, reti, variabili d'ambiente e Healthcheck per prevenire *race condition* all'avvio.

I servizi sono sviluppati in **Flask (Python)**, eseguendo parallelamente server REST e gRPC. La persistenza è affidata a **MariaDB** con ORM **SQLAlchemy** per sicurezza e pooling delle connessioni.

A. Strategia di Caching e Politica At-Most-Once

Elemento cardine dell'architettura è l'implementazione della semantica **At-Most-Once** tramite **Redis**. La scelta di questa tecnologia deriva dalla sua operatività in-memory (RAM), che assicura velocità estreme e bassa latenza. Inoltre, la funzionalità nativa di *Time-To-Live (TTL)* permette una gestione automatica e precisa della scadenza dei dati, mentre la sua natura di server centralizzato garantisce una cache condivisa e scalabile, ideale per l'ecosistema a microservizi.

L'utilizzo della cache è stato declinato in modo duale per rispondere a esigenze specifiche dei due moduli:

- **User Manager (Garanzia di Idempotenza):** In questo contesto, Redis è essenziale per la consistenza dei dati. Il sistema implementa un meccanismo di controllo basato sugli header HTTP X-REQUEST-ID (identificativo della richiesta) e X-CLIENT-ID (identificativo del client). Prima di eseguire operazioni critiche come la registrazione, il servizio verifica la presenza di questa coppia di chiavi in cache. Questo permette di intercettare eventuali *retry* di rete automatici, prevenendo l'esecuzione duplicata di codice sul database e garantendo che ogni operazione venga processata al massimo una volta.
- **Data Collector (Ottimizzazione delle Performance):** In questo modulo, l'uso di Redis è orientato all'efficienza computazionale e alla riduzione del carico, seppur il meccanismo di controllo basato sugli header HTTP X-REQUEST-ID e X-CLIENT-ID è uguale a quello implementato nello User Manager. La cache viene utilizzata per memorizzare i risultati di query onerose in termini di risorse dati e dati frequentemente richiesti (*hot data*). Questo approccio minimizza le letture sul database relazionale e riduce la latenza nelle risposte all'utente finale.

B. Circuit Breaker

L'architettura del sistema integra il pattern **Circuit Breaker** con l'obiettivo di garantire la resilienza e prevenire i cosiddetti *cascading failures* (fallimenti a cascata). Nello specifico, questo modulo agisce da intermediario per tutte le interazioni con il servizio esterno OpenSky.

Il meccanismo opera monitorando il tasso di errore delle chiamate: qualora il servizio OpenSky risulti irraggiungibile o instabile oltre una certa soglia, il circuito commuta nello stato **OPEN**. In questa fase, le richieste successive vengono bloccate preventivamente, restituendo un errore immediato senza saturare le risorse di rete.

Trascorso un periodo di *cooldown* (o *recovery*), il sistema transita nello stato **HALF-OPEN**, permettendo l'inoltro di un numero limitato di richieste di prova. Se queste hanno esito positivo, il circuito viene ripristinato (**CLOSED**), tornando alla normale operatività; in caso contrario, il circuito si riapre, ricominciando il ciclo di protezione.

C. Comunicazione Asincrona (Confluent Kafka)

Per garantire il disaccoppiamento tra il layer di acquisizione dati e la logica di notifica, l'architettura adotta un pattern di comunicazione asincrona basato su Confluent Kafka. Questa scelta architetturale permette di gestire picchi di traffico e di isolare i processi di elaborazione (Alert System) e distribuzione (Notifier) dal ciclo di vita del Data Collector.

Il flusso dei messaggi è orchestrato attraverso una pipeline a due stadi, che coinvolge due topic distinti e tre microservizi:

1. Ingestione Dati (Topic `to-alert-system`) Il Data Collector Service agisce come *Producer* primario. Al termine del fetch dei dati da OpenSky, per ogni aeroporto monitorato, costruisce un payload JSON contenente:

- Il numero attuale di voli rilevati.
- Le soglie di allarme configurate dall'utente (`high_value` e `low_value`).
- L'identificativo dell'utente (email).

Questo messaggio viene pubblicato sul topic `to-alert-system`. In questa fase non viene eseguita alcuna logica di controllo; il Data Collector si limita a segnalare lo "stato attuale" del sistema.

2. Elaborazione e Filtraggio (Alert System) L'Alert System agisce come *Consumer* sul topic `to-alert-system`. La sua funzione è puramente logica: per ogni messaggio ricevuto, confronta il numero di voli con le soglie presenti nel payload.

- Logica di Business: Se `voli_attuali > high_value` (traffico eccessivo) oppure `voli_attuali < low_value` (traffico scarso), il sistema determina la necessità di un avviso.
- In caso di attivazione della regola, l'Alert System diventa a sua volta *Producer*, pubblicando un nuovo messaggio sul topic `to-notify`. I messaggi che non violano le soglie vengono scartati, filtrando il rumore a monte del sistema di notifica.

3. Distribuzione Notifiche (Topic `to-notify`) L'Alert Notifier System è il consumatore finale della pipeline. Sottoscrive il topic `to-notify`, che contiene esclusivamente eventi "confermati" che richiedono un'azione. Preleva il messaggio e si occupa dell'invio effettivo dell'email all'utente finale tramite protocollo SMTP.

Attualmente, i *Consumer* sono configurati con un `BATCH_SIZE = 1`. Questa scelta privilegia la bassa latenza (Real-Time) rispetto al throughput: ogni messaggio viene prelevato, processato e committato singolarmente, garantendo che l'utente riceva l'alert nell'istante esatto in cui il dato viene analizzato.

Evoluzioni Future: Sebbene la configurazione attuale sia ottimale per la reattività, in scenari di produzione ad alto carico la *Batch Size* è un parametro chiave per la scalabilità orizzontale. Nelle versioni future dell'applicazione, si prevede di:

1. Aumentare il *Batch Size* per processare blocchi di messaggi simultaneamente, riducendo l'overhead di rete su Kafka.
2. Introdurre logiche di aggregazione (Debouncing) nell'Alert System: questo eviterà di inviare "*n email per n aggiornamenti consecutivi*", raggruppando le variazioni di traffico in un unico report periodico per migliorare l'esperienza utente ed evitare spam.

A supporto della gestione operativa dei flussi dati, l'architettura integra **Kafka UI**, un'interfaccia web open-source progettata per semplificare l'interazione visiva con il cluster Kafka. L'adozione di questo strumento si è rivelata essenziale durante le fasi di sviluppo e testing per ispezionare in tempo reale l'effettiva creazione e il popolamento dei topic critici, come `to-alert-system` e `to-notify`. Nello specifico, la dashboard ha consentito di analizzare direttamente la struttura dei payload JSON scambiati tra i microservizi, validando la correttezza dei dati senza la necessità di ricorrere a complessi comandi da terminale. Inoltre, lo strumento ha offerto una visibilità immediata sullo stato dei Consumer Group, facilitando l'identificazione di eventuali ritardi (lag) nel processamento dei messaggi e garantendo che il sistema di alert mantenesse le performance attese senza accumulare code impreviste.

D. SSL

Nell'ottica di adottare le best practice di sicurezza sin dalle prime fasi di sviluppo, l'architettura è stata progettata per garantire la cifratura del traffico in transito tramite protocollo HTTPS. Per l'ambiente di esecuzione locale, non disponendo di un dominio pubblico validato da un'autorità di certificazione (CA), si è optato per la generazione manuale di certificati SSL autofirmati (Self-Signed Certificates).

Questa configurazione è applicata a livello dell'API Gateway, che agisce come terminatore SSL: il Gateway espone la porta sicura 443 verso l'esterno, gestendo l'handshake crittografico con il client (Postman o Browser) e decifrando le richieste prima di inoltrarle ai microservizi interni su rete privata. Sebbene questa soluzione comporti la visualizzazione di avvisi di sicurezza nei client standard (poiché l'autorità emittente non è riconosciuta pubblicamente), essa permette di simulare fedelmente uno scenario di produzione, validando le configurazioni dei web server e garantendo che nessun dato sensibile, come le credenziali utente o i token, viaggi in chiaro sulla rete. In una futura fase di deploy in produzione, questa infrastruttura è predisposta per la sostituzione trasparente dei certificati manuali con quelli validi rilasciati da autorità.

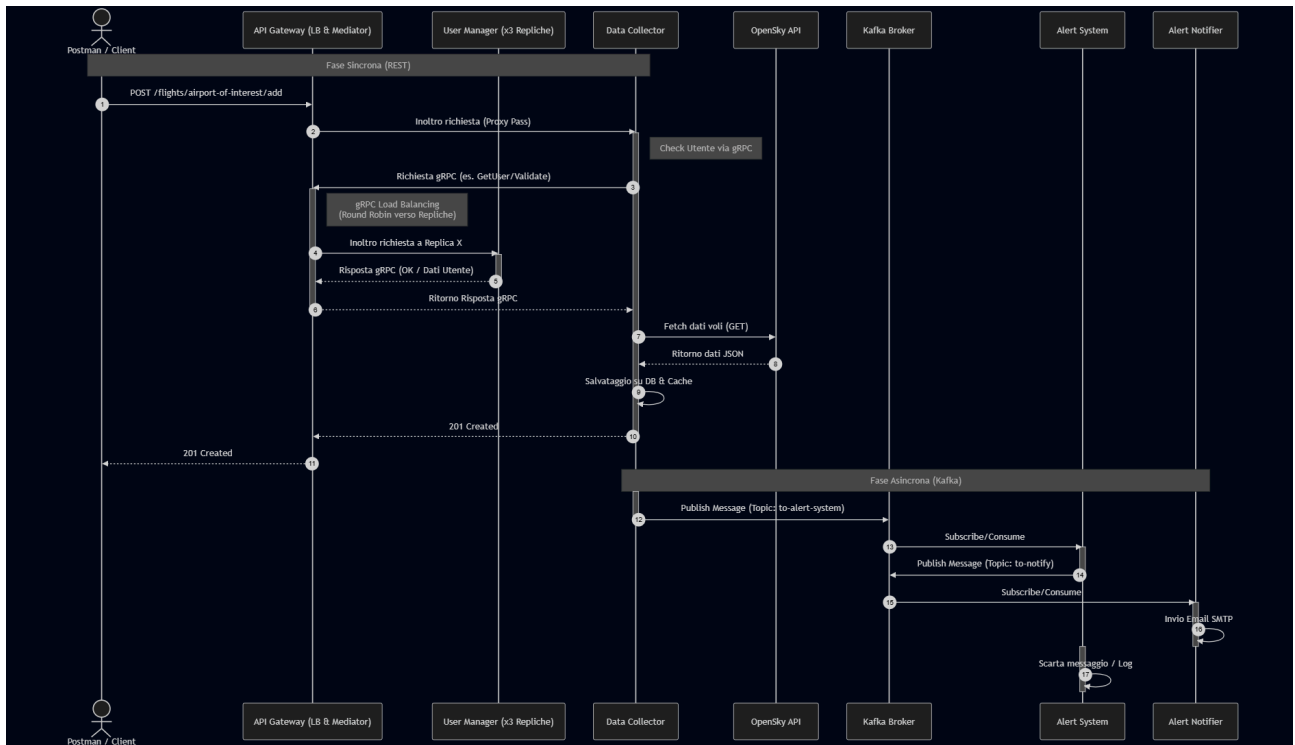
4. Topologia di Rete

La sicurezza è garantita dalla segmentazione delle reti Docker:

- **User-Network:** Isolamento per User Manager, User DB e User Cache.
 - **Data-Network:** Isolamento per Data Collector, Data DB e Data Cache.
 - **Gateway-Network:** Permette la comunicazione tra Api Gateway, Data Collector e User Manager.
 - **Kafka-Network:** Permette la comunicazione tra Data Collector, Alert System, Alert Notifier System, Broker, Kafka UI.
-

5. Diagramma delle interazioni

Viene di seguito illustrato il diagramma di sequenza relativo allo scenario "Add Airports of Interest". Tale casistica è stata selezionata poiché rappresenta il flusso più articolato dell'architettura: essa orchestra l'intero stack tecnologico, integrando comunicazioni sincrone (REST e mediazione gRPC) per l'acquisizione dati con la pipeline asincrona su Kafka per la gestione degli alert.



6. Istruzioni per l'esecuzione del codice

Vedere README.md presente nella repository GitHub.